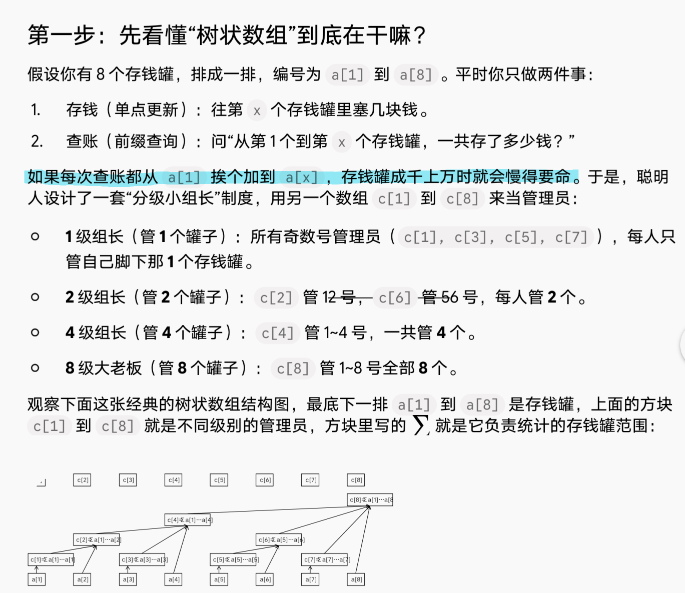
树状数组

树状数组就是一种通过“分层”+“排重”来节省时间和内存的一种算法



第一步：先看懂“树状数组”到底在干嘛？

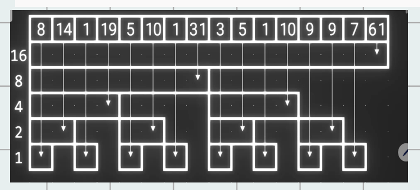
假设你有 8 个存钱罐，排成一排，编号为 `a[1]` 到 `a[8]`。平时你只做两件事：

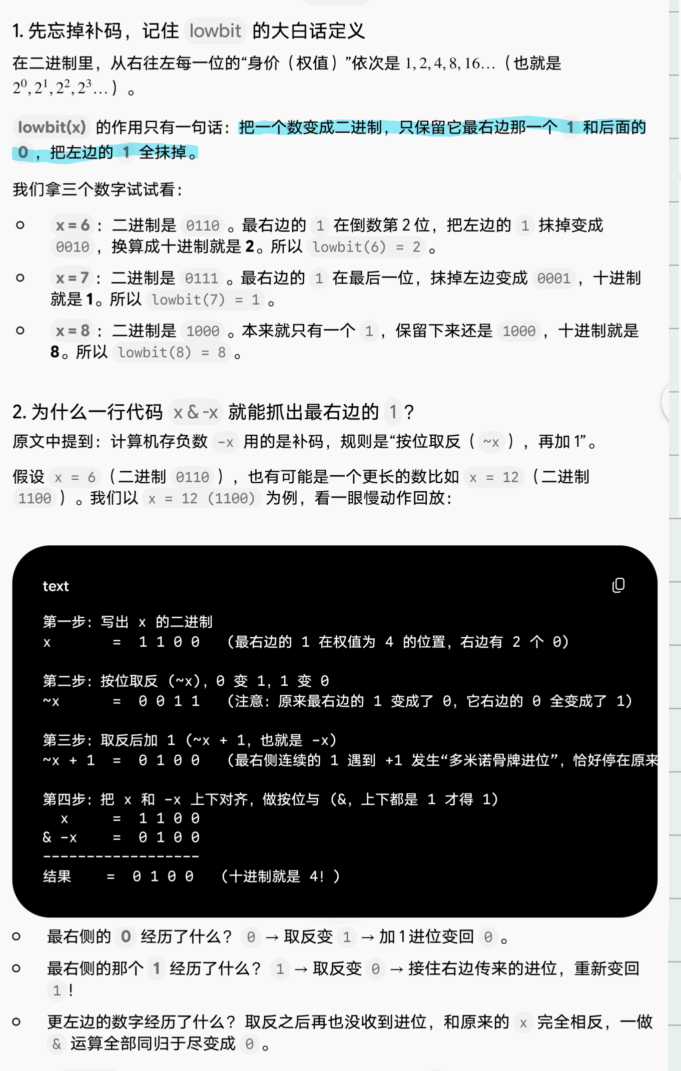
1. 存钱（单点更新）：往第 `x` 个存钱罐里塞几块钱。
2. 查账（前缀查询）：问“从第 1 个到第 `x` 个存钱罐，一共存了多少钱？”

如果每次查账都从 `a[1]` 挨个加到 `a[x]`，存钱罐成千上万时就会慢得要命。于是，聪明人设计了一套“分级小组长”制度，用另一个数组 `c[1]` 到 `c[8]` 来当管理员：

- **1** 级组长（管 **1** 个罐子）：所有奇数号管理员（`c[1]`, `c[3]`, `c[5]`, `c[7]`），每人只管自己脚下那 **1** 个存钱罐。
- **2** 级组长（管 **2** 个罐子）：`c[2]` 管 ~~12 号，~~ `c[6]` ~~管 56~~ 号，每人管 **2** 个。
- **4** 级组长（管 **4** 个罐子）：`c[4]` 管 1~4 号，一共管 **4** 个。
- **8** 级大老板（管 **8** 个罐子）：`c[8]` 管 1~8 号全部 **8** 个。

观察下面这张经典的树状数组结构图，最底下一排 `a[1]` 到 `a[8]` 是存钱罐，上面的方块 `c[1]` 到 `c[8]` 就是不同级别的管理员，方块里写的 $\sum$ 就是它负责统计的存钱罐范围：





第二步：拆解原文第 1 节—— lowbit 到底是什么？

1. 先忘掉补码，记住 lowbit 的大白话定义

在二进制里，从右往左每一位的“身价（权值）”依次是 $1, 2, 4, 8, 16\ldots$（也就是 $2^0, 2^1, 2^2, 2^3\ldots$）。

lowbit(x) 的作用只有一句话：把一个数变成二进制，只保留它最右边那一个 1 和后面的 0，把左边的 1 全抹掉。

我们拿三个数字试试看：

- **x = 6**：二进制是 `0110`。最右边的 `1` 在倒数第 2 位，把左边的 `1` 抹掉变成 `0010`，换算成十进制就是 **2**。所以 `lowbit(6) = 2`。
- **x = 7**：二进制是 `0111`。最右边的 `1` 在最后一位，抹掉左边变成 `0001`，十进制就是 **1**。所以 `lowbit(7) = 1`。
- **x = 8**：二进制是 `1000`。本来就只有一个 `1`，保留下来还是 `1000`，十进制就是 **8**。所以 `lowbit(8) = 8`。

2. 为什么一行代码 x & -x 就能抓出最右边的 1？

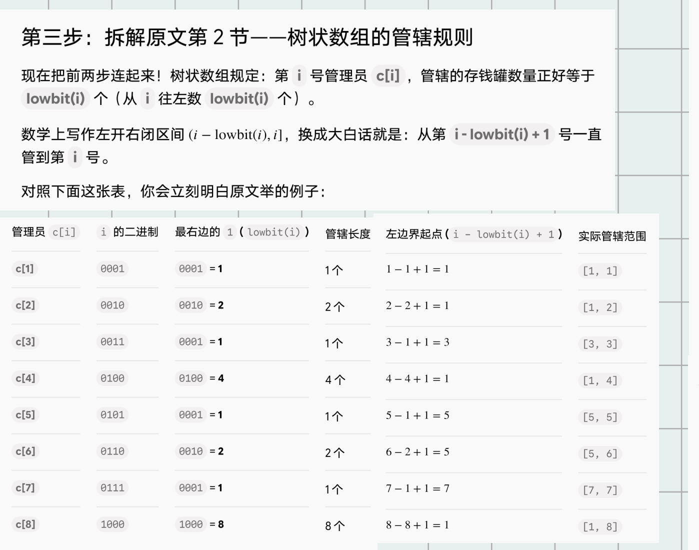
原文中提到：计算机存负数 `-x` 用的是补码，规则是“按位取反（`~x`），再加 1”。

假设 `x = 6`（二进制 `0110`），也有可能是一个更长的数比如 `x = 12`（二进制 `1100`）。我们以 `x = 12 (1100)` 为例，看一眼慢动作回放：

```text
第一步：写出 x 的二进制
x        =  1 1 0 0   (最右边的 1 在权值为 4 的位置，右边有 2 个 0)

第二步：按位取反 (~x)，0 变 1，1 变 0
~x       =  0 0 1 1   (注意：原来最右边的 1 变成了 0，它右边的 0 全变成了 1)

第三步：取反后加 1 (~x + 1，也就是 -x)
~x + 1   =  0 1 0 0   (最右侧连续的 1 遇到 +1 发生“多米诺骨牌进位”，恰好停在原来

第四步：把 x 和 -x 上下对齐，做按位与 (&，上下都是 1 才得 1)
  x      =  1 1 0 0
& -x     =  0 1 0 0
------------------
结果     =  0 1 0 0   (十进制就是 4！)
```

- 最右侧的 **0** 经历了什么？`0` → 取反变 `1` → 加 1 进位变回 `0`。
- 最右侧的那个 **1** 经历了什么？`1` → 取反变 `0` → 接住右边传来的进位，重新变回 `1`！
- 更左边的数字经历了什么？取反之后再也没收到进位，和原来的 `x` 完全相反，一做 `&` 运算全部同归于尽变成 `0`。

所以，`x & -x` 就像一个精准的筛子，只把最右边的那个 `1` 筛了出来。



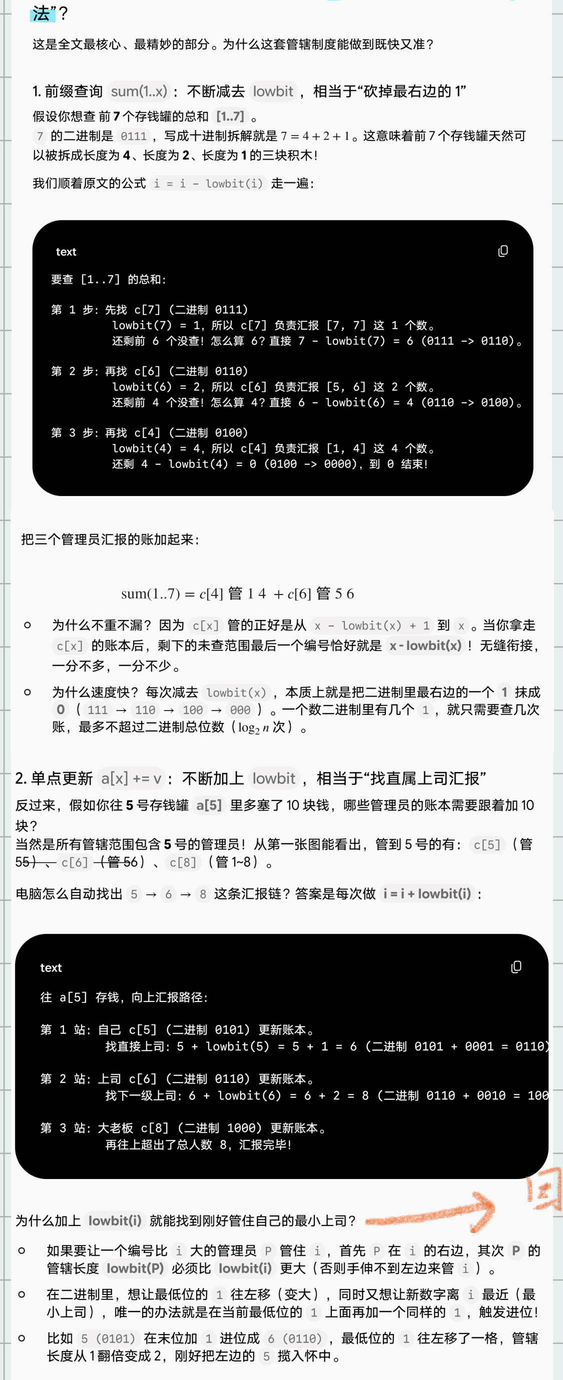
第三步：拆解原文第 2 节——树状数组的管辖规则

现在把前两步连起来！树状数组规定：第 **i** 号管理员 **c[i]**，管辖的存钱罐数量正好等于 **lowbit(i)** 个（从 **i** 往左数 **lowbit(i)** 个）。

数学上写作左开右闭区间 $(i - \text{lowbit}(i), i]$，换成大白话就是：从第 **i - lowbit(i) + 1** 号一直管到第 **i** 号。

对照下面这张表，你会立刻明白原文举的例子：

管理员 `c[i]`	`i` 的二进制	最右边的 `1`（`lowbit(i)`）	管辖长度	左边界起点（`i - lowbit(i) + 1`）	实际管辖范围
c[1]	`0001`	`0001` = **1**	1个	$1-1+1=1$	`[1, 1]`
c[2]	`0010`	`0010` = **2**	2个	$2-2+1=1$	`[1, 2]`
c[3]	`0011`	`0001` = **1**	1个	$3-1+1=3$	`[3, 3]`
c[4]	`0100`	`0100` = **4**	4个	$4-4+1=1$	`[1, 4]`
c[5]	`0101`	`0001` = **1**	1个	$5-1+1=5$	`[5, 5]`
c[6]	`0110`	`0010` = **2**	2个	$6-2+1=5$	`[5, 6]`
c[7]	`0111`	`0001` = **1**	1个	$7-1+1=7$	`[7, 7]`
c[8]	`1000`	`1000` = **8**	8个	$8-8+1=1$	`[1, 8]`



第四步：拆解原文第 3 节——为什么查账用“减法”，更新用“加法”？

这是全文最核心、最精妙的部分。为什么这套管辖制度能做到既快又准？

1. 前缀查询 sum(1..x)：不断减去 lowbit，相当于“砍掉最右边的 1”

假设你想查 前 **7** 个存钱罐的总和 **[1..7]**。
`7` 的二进制是 `0111`，写成十进制拆解就是 $7 = 4 + 2 + 1$。这意味着前 7 个存钱罐天然可以被拆成长度为 **4**、长度为 **2**、长度为 **1** 的三块积木！

我们顺着原文的公式 `i = i - lowbit(i)` 走一遍：

```text
要查 [1..7] 的总和：

第 1 步：先找 c[7] (二进制 0111)
        lowbit(7) = 1，所以 c[7] 负责汇报 [7, 7] 这 1 个数。
        还剩前 6 个没查！怎么算 6？直接 7 - lowbit(7) = 6 (0111 -> 0110)。

第 2 步：再找 c[6] (二进制 0110)
        lowbit(6) = 2，所以 c[6] 负责汇报 [5, 6] 这 2 个数。
        还剩前 4 个没查！怎么算 4？直接 6 - lowbit(6) = 4 (0110 -> 0100)。

第 3 步：再找 c[4] (二进制 0100)
        lowbit(4) = 4，所以 c[4] 负责汇报 [1, 4] 这 4 个数。
        还剩 4 - lowbit(4) = 0 (0100 -> 0000)，到 0 结束！
```

把三个管理员汇报的账加起来：

$$\text{sum}(1..7) = c[4]\ 管\ 1\ 4\ + c[6]\ 管\ 5\ 6$$

- 为什么不重不漏？因为 `c[x]` 管的正好是从 `x - lowbit(x) + 1` 到 `x`。当你拿走 `c[x]` 的账本后，剩下的未查范围最后一个编号恰好就是 **x - lowbit(x)**！无缝衔接，一分不多，一分不少。
- 为什么速度快？每次减去 `lowbit(x)`，本质上就是把二进制里最右边的一个 **1** 抹成 **0**（`111` → `110` → `100` → `000`）。一个数二进制里有几个 `1`，就只需要查几次账，最多不超过二进制总位数（$\log_2 n$ 次）。

2. 单点更新 a[x] += v：不断加上 lowbit，相当于“找直属上司汇报”

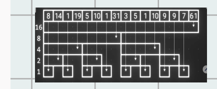
反过来，假如你往 **5** 号存钱罐 **a[5]** 里多塞了 10 块钱，哪些管理员的账本需要跟着加 10 块？
当然是所有管辖范围包含 **5** 号的管理员！从第一张图能看出，管到 5 号的有：`c[5]`（管 ~~55）、~~ `c[6]` ~~（管 56~~）、`c[8]`（管 1~8）。

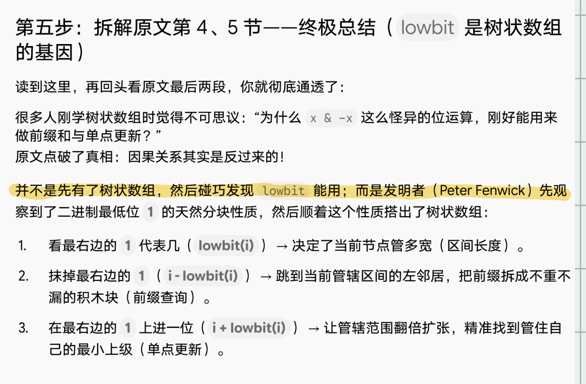
电脑怎么自动找出 `5` → `6` → `8` 这条汇报链？答案是每次做 **i = i + lowbit(i)**：

```text
往 a[5] 存钱，向上汇报路径：

第 1 站：自己 c[5] (二进制 0101) 更新账本。
        找直接上司：5 + lowbit(5) = 5 + 1 = 6 (二进制 0101 + 0001 = 0110)

第 2 站：上司 c[6] (二进制 0110) 更新账本。
        找下一级上司：6 + lowbit(6) = 6 + 2 = 8 (二进制 0110 + 0010 = 100

第 3 站：大老板 c[8] (二进制 1000) 更新账本。
        再往上超出了总人数 8，汇报完毕！
```

为什么加上 **lowbit(i)** 就能找到刚好管住自己的最小上司？ → 因为每一层的个数均是上一层的一半

- 如果要让一个编号比 `i` 大的管理员 `P` 管住 `i`，首先 `P` 在 `i` 的右边，其次 **P** 的管辖长度 **lowbit(P)** 必须比 **lowbit(i)** 更大（否则手伸不到左边来管 `i`）。
- 在二进制里，想让最低位的 `1` 往左移（变大），同时又想让新数字离 `i` 最近（最小上司），唯一的办法就是在当前最低位的 `1` 上面再加一个同样的 `1`，触发进位！
- 比如 `5 (0101)` 在末位加 `1` 进位成 `6 (0110)`，最低位的 `1` 往左移了一格，管辖长度从 1 翻倍变成 2，刚好把左边的 `5` 揽入怀中。





第五步：拆解原文第 4、5 节——终极总结（lowbit 是树状数组的基因）

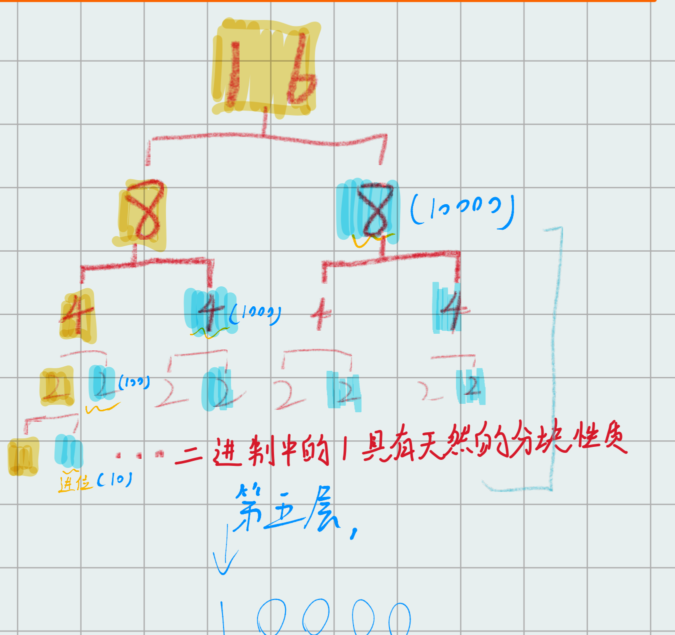
读到这里，再回头看原文最后两段，你就彻底通透了：

很多人刚学树状数组时觉得不可思议：“为什么 `x & -x` 这么怪异的位运算，刚好能用来做前缀和与单点更新？”
原文点破了真相：因果关系其实是反过来的！

并不是先有了树状数组，然后碰巧发现 `lowbit` 能用；而是发明者（Peter Fenwick）先观察到了二进制最低位 `1` 的天然分块性质，然后顺着这个性质搭出了树状数组：

1. 看最右边的 `1` 代表几（**lowbit(i)**）→ 决定了当前节点管多宽（区间长度）。
2. 抹掉最右边的 `1`（**i - lowbit(i)**）→ 跳到当前管辖区间的左邻居，把前缀拆成不重不漏的积木块（前缀查询）。
3. 在最右边的 `1` 上进一位（**i + lowbit(i)**）→ 让管辖范围翻倍扩张，精准找到管住自己的最小上级（单点更新）。

2, 2^2, 2^3, …, 2^n
10_2, 100_2, 1000_2, …, $1\cdots0_2$ (n个0)
二进制中，1后面有k个0，这个数就是2^k



……二进制中的1具有天然的分块性质

第五层，

10000

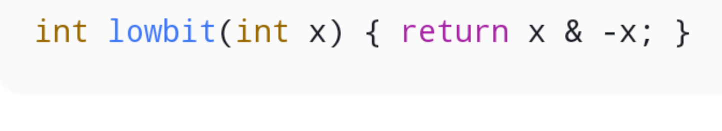
按二进制末尾 0 的个数分层，每层就是“奇数 × 2^k”

省略的是每一层中的偶数项

省略的是每一层中的偶数项（偶数项不在当前层级）

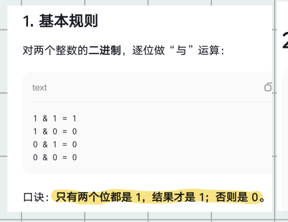
---

lowbit: x & -x



```c
int lowbit(int x) { return x & -x; }
```

“&”是“按位与(bitwise AND)”运算符

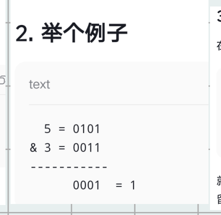


1. 基本规则

对两个整数的**二进制**，逐位做“与”运算：

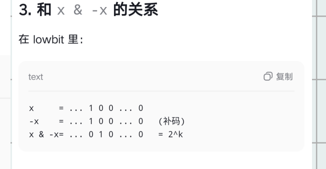
```text
1 & 1 = 1
1 & 0 = 0
0 & 1 = 0
0 & 0 = 0
```

口诀：**只有两个位都是 1，结果才是 1；否则是 0。**



2. 举个例子

```text
  5 = 0101
& 3 = 0011
-----------
      0001  = 1
```



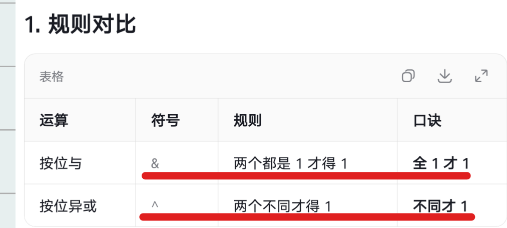
3. 和 x & -x 的关系

在 lowbit 里：

```text
x     = ... 1 0 0 ... 0
-x    = ... 1 0 0 ... 0   (补码)
x & -x= ... 0 1 0 ... 0   = 2^k
```

就是逐位做与，只有最低位 1 那一位两边都是 1，所以保留下来，其余位至少有一个是 0，被清掉。

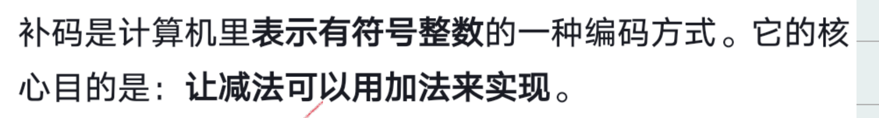
→ “&”与“^”(按位异或)都是逐位运算，但规则完全不同



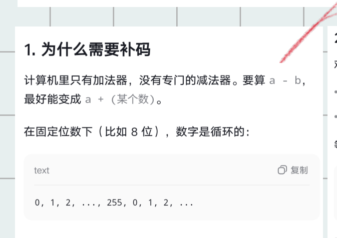
1. 规则对比

运算	符号	规则	口诀
按位与	&	两个都是 1 才得 1	**全 1 才 1**
按位异或	^	两个不同才得 1	**不同才 1**

“-x”是x的“补码” → 何为补码?



补码是计算机里**表示有符号整数**的一种编码方式。它的核心目的是：**让减法可以用加法来实现。**



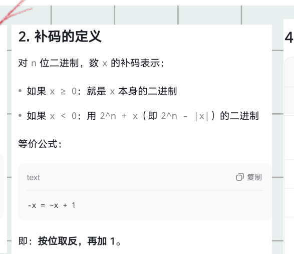
1. 为什么需要补码

计算机里只有加法器，没有专门的减法器。要算 `a - b`，最好能变成 `a + (某个数)`。

在固定位数下（比如 8 位），数字是循环的：

```text
0, 1, 2, ..., 255, 0, 1, 2, ...
```

如果 `b` 加上一个数等于 0（模 256），那这个数就是 `-b`。



2. 补码的定义

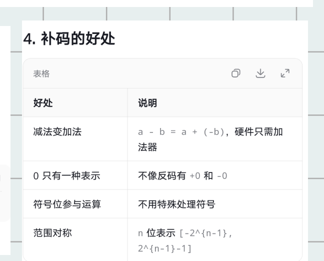
对 `n` 位二进制，数 `x` 的补码表示：

- 如果 `x ≥ 0`：就是 `x` 本身的二进制
- 如果 `x < 0`：用 `2^n + x`（即 `2^n - |x|`）的二进制

等价公式：

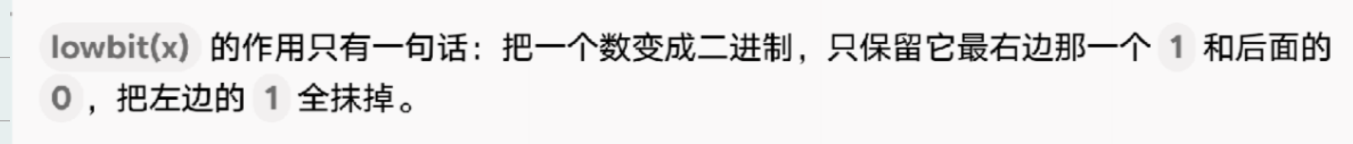
```text
-x = ~x + 1
```

即：**按位取反，再加 1**。

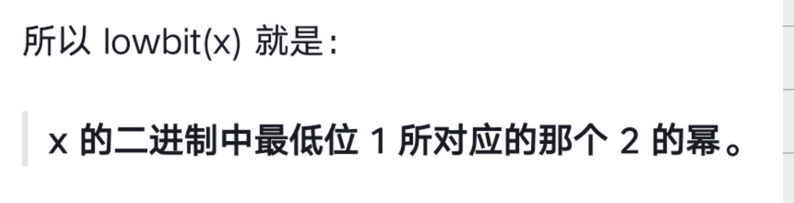


4. 补码的好处

好处	说明
减法变加法	`a - b = a + (-b)`，硬件只需加法器
0 只有一种表示	不像反码有 `+0` 和 `-0`
符号位参与运算	不用特殊处理符号
范围对称	`n` 位表示 `[-2^{n-1}, 2^{n-1}-1]`



lowbit(x) 的作用只有一句话：把一个数变成二进制，只保留它最右边那一个 `1` 和后面的 `0`，把左边的 `1` 全抹掉。

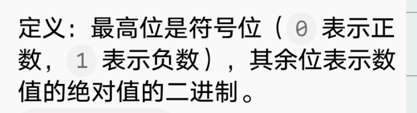


所以 lowbit(x) 就是：

x 的二进制中最低位 1 所对应的那个 2 的幂。

补码如何算得?

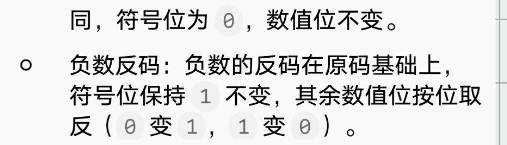
正码：



定义：最高位是符号位（`0` 表示正数，`1` 表示负数），其余位表示数值的绝对值的二进制。

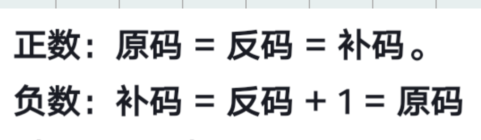
5的原码=0000 0101
0表示正，1表示负
由此可得-5的原码为1000 0101

反码：



- 正数反码：正数的反码与原码完全相同，符号位为 `0`，数值位不变。
- 负数反码：负数的反码在原码基础上，符号位保持 `1` 不变，其余数值位按位取反（`0` 变 `1`，`1` 变 `0`）。

补码：



正数：原码 = 反码 = 补码。
负数：补码 = 反码 + 1 = 原码（符号位外）取反加一。 ~x +